P5

Comprobaciones anti-debug y empaquetado — Criterios y solución docente

Parte I · RA5 · 2 h

Universidad de Almería · Edición de 27 de septiembre de 2026

Documento: Criterios y solución de referencia
Asignatura: Ingeniería Inversa y Análisis Forense Digital
Uso: Laboratorio docente en entorno controlado y sobre las muestras indicadas
Alumno/a: _____ Grupo: _____

Material reservado para corrección docente

Este documento contiene criterios de corrección y resultados de referencia. No debe incorporarse al paquete evaluable que recibe el alumnado.

Índice

1. Solución de referencia **2**

2. Desarrollo de referencia **2**
2.1. Anti-debug 2
2.2. Empaquetado 2

3. Tabla docente de aceptación **3**

4. Solución UPX-01 1.0 — 28 de septiembre de 2026 **3**
4.1. Interpretación de estructura y cadenas 3
4.2. Comportamiento medido 3

> **Estado actual: UPX-01 1.0 validado.** La actualización del 28 de septiembre de 2026, al final del documento, incorpora la muestra empaquetada y sus resultados medidos. Sustituye el estado UPX pendiente de la ficha histórica, que se conserva. El bloque anti-debug mantiene sus controles propios.

Material para el profesorado. No incluir en el paquete evaluable del alumnado. No se tratan como secretos los fuentes o soluciones publicados en el repositorio.

1 Solución de referencia

En condiciones habituales, `PTRACE_TRACEME` tiene éxito sin tracer y falla bajo GDB. Solicita trazado por el padre, no auto-trazado. Restricciones de seguridad también pueden causar fallo. El mensaje de la muestra simplifica la interpretación; se debe contrastar el retorno y reconocer que no es una prueba universal de depuración.

Bajo GDB, `catch syscall ptrace` permite parar a la entrada y a la salida. Solo tras la parada de salida se modifica el retorno en `rax` en Linux x86-64, si se opta por ese método. También sirve detenerse justo tras la llamada en `main` y corregir el valor antes de la comprobación. Se registra la instrucción del build usado, sin una secuencia ciega de `continue` que no distinga entrada y salida.

La recuperación UPX se comprueba con `cmp` (código 0 significa bytes idénticos). Si no coincide, se investigan opciones/versiones y estructura; no se inventa igualdad. Entropía alta puede ser compresión/cifrado u otros datos; no demuestra malware ni aparece necesariamente una gráfica con bordes perfectos. El número de secciones no tiene un valor canónico para todos los builds. El informe explica esas diferencias.

Rúbrica y pesos: evaluación de Parte I.

2 Desarrollo de referencia

2.1 Anti-debug

La respuesta debe identificar la llamada, sus argumentos, retorno, comparación y salto. Un fallo de `ptrace` es compatible con GDB, pero también con restricciones. La intervención válida registra el retorno original y cambia el estado justo antes de la decisión. Cambiar flags que luego sobrescribe un `cmp` no sirve.

En el contenedor de validación actual, incluso la ejecución directa imprimió el mensaje de detección y retornó 1; GDB no pudo trazar por política del entorno. Esto es un control negativo útil: confirma que no debe fijarse como solución universal «sin GDB siempre tiene éxito». La práctica debe ejecutarse en la VM validada.

2.2 Empaquetado

Se esperan tres objetos inventariados. El empaquetado debe diferir del original. `upx -t` comprueba la estructura que reconoce UPX. Tras `upx -d`, `cmp` establece si el recuperado coincide byte a byte. Si no coincide, se conserva el resultado y se revisan versión/opciones; no se cambia por una captura preparada.

UPX no estaba instalado en el entorno de revisión documental, de modo que no se publican tamaños, hashes ni igualdad ficticios. El docente completa la tabla al validar la VM de aula.

3 Tabla docente de aceptación

Control	Evidencia mínima
sin/bajo GDB	stdout, retorno, política y versión
intervención	localizador, antes/después y control nuevo
empaquetado	comando, log, hash y estructura
recuperación	comando, hash y código de `cmp`
comportamiento	mismas entradas y efectos delimitados

La evaluación premia la interpretación de diferencias, no obtener necesariamente un hash igual con cualquier versión.

4 Solución UPX-01 1.0 — 28 de septiembre de 2026

La revisión anterior carecía de UPX; este bloque ya se ha ejecutado con UPX 4.2.2, paquete upx-ucl 4.2.2-3. Se conserva el texto histórico. Los resultados corresponden a esta edición, identificada por el manifiesto. Fuente, compilación, operaciones, informes ELF y ejecuciones están en el paquete docente. La validación delimita su alcance.

Objeto	Bytes reales	Entry ELF	Program headers	Section headers
original	14472	0x1150	13	29
empaquetado	5844	0x58a0	3	0
recuperado	14472	0x1150	13	29

SHA-256 del original y del recuperado:

```
125c09bcfecf485febfe7cb74787dc84000263edc4f798d1c12905067f93eaf0
```

SHA-256 del empaquetado:

```
184e21256a3816e44d0204070b376811eac295c0f179d0dda4ee1bf56498f079
```

upx -9, -t, -l y -d retornaron 0; cmp -s retornó 0. La reducción del tamaño real es (14472-5844)/14472, aproximadamente 59,62 %. original y recuperado son idénticos byte a byte. El log de -d muestra 23231 en su columna de progreso; el fichero final mide 14472 bytes. La corrección exige medirlo y conservar el log, no transformar esa columna en una respuesta de tamaño que contradiga el archivo producido.

4.1 Interpretación de estructura y cadenas

El empaquetado no tiene tabla de secciones, pero conserva dos segmentos LOAD y un GNU_STACK: tres program headers. Su entry ELF apunta al arranque del stub. La ausencia de secciones no equivale a ausencia de código o de imagen cargable. Son VA ELF; no se interpretan como offsets ni direcciones absolutas con ASLR.

La cadena R3v3rs1ng_UAL permanece literalmente en los tres archivos. Es una observación concreta: empaquetar no cifra necesariamente cada cadena ni garantiza ocultación. No se penaliza que strings la encuentre en esta muestra.

file describe la envoltura como statically linked, sin cabeceras de sección. La aplicación original recuperada sigue teniendo sus dependencias dinámicas: esa descripción de la envoltura no prueba que hayan desaparecido. El manifiesto conserva los requisitos glibc leídos del original.

4.2 Comportamiento medido

Se probaron diez entradas sobre las tres representaciones, 30 procesos en total:

Entrada	**Retorno en los tres objetos**	**Mensaje tras la línea aritmética**
sin argumentos	1	uso con argv[0]
vacía; abc; 13 letras A	1	Contraseña incorrecta.
R3v3rs1ng_UAX	1	Contraseña incorrecta.
R3v3rs1ng_UAL	0	Contraseña correcta.
válida con argumento extra	0	Contraseña correcta.
válida con salto final	1	Contraseña incorrecta.
contraseña con ñ; 4096 letras A	1	Contraseña incorrecta.

La primera línea común es retorno=42 if=1 bucle=45 llamada=6. stderr estuvo vacío. verificar.py fija argv[0]=muestra en los tres procesos para comparar exactamente el mensaje de uso; no elimina ni normaliza otras diferencias. El informe conserva las entradas y los textos completos. Las pruebas del guion con argumento no requieren igualar argv[0] porque sus mensajes no lo incluyen.

Un archivo de control y el inventario de cada directorio temporal permanecieron intactos. No se observaron otras carpetas ni se trazaron todas las llamadas al sistema, por lo que no se afirma ausencia universal de efectos. Los binarios no cambiaron.

La reconstrucción autónoma desde el archivo docente produjo los mismos tres binarios con la misma toolchain y repitió los controles. Los resultados no prueban equivalencia para toda entrada ni para otra versión de UPX. Esta validación completa empaquetado; el bloque ptrace/GDB conserva sus requisitos y controles propios.